

Aditya Sharma
IIT Delhi

ee1230692@iitd.ac.in

Harshik Bains
IIT Delhi

ee1230425@iitd.ac.in

Abstract

Classification tasks often involve samples with varying levels of confidence, where some examples are consistently classified with high confidence while others remain challenging. Traditional neural networks process all samples through identical pathways, potentially missing opportunities to adapt to these natural confidence variations. We present a confidence-based neural tree framework that incorporates prediction certainty into the classification structure. Our approach employs a shared neural backbone with sequential classification heads that expand from binary to multi-class outputs across iterations. After each classification stage, we re-label samples based on both their true class and previous prediction confidence, creating a tree-like structure in the label space where different indices reflect classification certainty. Rather than routing samples through different paths, our model assigns them to different positions in the expanded label space based on confidence patterns. During inference, predictions are made by aggregating specific probability values across all classification heads. The framework reveals how samples distribute across confidence patterns while maintaining competitive accuracy on benchmark datasets, potentially offering additional context for understanding model behavior on samples with different confidence characteristics.

1. Introduction

Neural networks have achieved impressive performance across various domains, but they typically process all samples through identical pathways despite natural variations in classification confidence. Research shows that some samples consistently receive high-confidence predictions while others consistently produce low-confidence outputs across different model architectures [16].

Standard deep learning architectures [8, 13] apply the same operations to all inputs, potentially missing opportunities to adapt to varying confidence levels. While ensemble methods like Gradient Boosting [1, 5] give greater weight to misclassified examples, they don't structurally differentiate between high and low confidence patterns in their architecture.

Previous work has explored uncertainty through Bayesian approaches [6], calibration techniques [7], and confidence estimation [9], but typically focuses on measuring or improving uncertainty estimates rather than incorporating confidence directly into the classification structure itself.

We propose Neural Confidence Trees (NCT), a framework that integrates prediction confidence into the classification process through a hierarchical label structure. Our approach uses a shared neural backbone with sequential classification heads that operate across successive iterations. The NCT framework consists of three core components: (1) a shared neural backbone that maintains feature learning capacity across iterations, (2) sequential classification heads that progressively expand from binary to multi-class outputs, and (3) a confidence-based re-labeling mechanism that creates tree-structured label spaces where sample positions reflect their previous confidence patterns. Unlike traditional decision trees [2, 15] that use predefined splits on raw features, NCT leverages learned neural representations while organizing the classification problem in a hierarchical manner.

Experiments on benchmark datasets demonstrate that this confidence-based approach achieves competitive performance while revealing how samples distribute across different confidence patterns in the expanded label space.

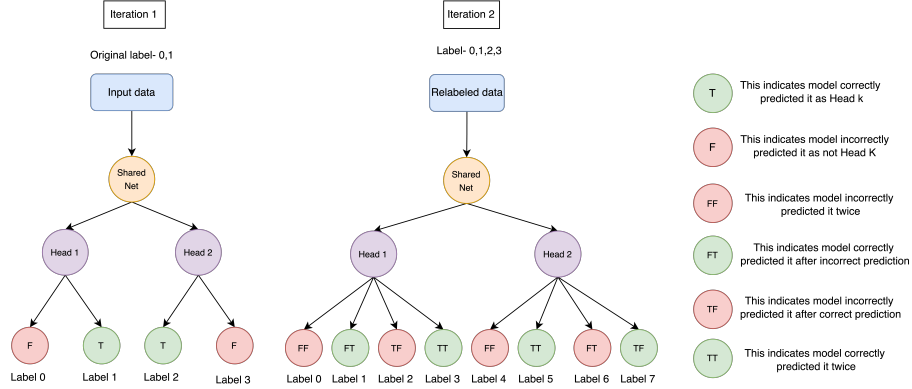


Figure 1: NCT architecture showing label expansion across iterations. The shared backbone feeds multiple classification heads, progressively expanding labels from binary (Iteration 1) to quaternary (Iteration 2).

2. Related Work

Confidence and uncertainty estimation in neural networks have been widely explored, with a focus on understanding how models handle example difficulty and prediction reliability. Studies on forgetting dynamics and training trajectories show that certain inputs consistently elicit low-confidence or unstable predictions [16]. To address this, Bayesian methods like Monte Carlo dropout [6] and post-hoc calibration techniques [7] aim to align predicted probabilities with actual correctness. However, these approaches treat uncertainty as a diagnostic tool rather than a structural element in the classification process. Related frameworks in conditional computation [4, 10] and ensemble-based methods like Random Forests [1] and Gradient Boosting [5] adapt to difficulty by varying computation or weighting, but they do not modify the label space or model confidence evolution directly.

Decision trees such as CART [2] and C4.5 [15] offer transparent decision rules but struggle with generalization on complex tasks. Neural-tree hybrids including Deep Neural Decision Forests [12], Conditional Networks [11], and Deep Neural Decision Trees [18] introduce differentiable branching based on learned features but do not incorporate confidence into their structural logic. Our approach differs by embedding prediction certainty directly into a progressively expanding label space. Through a shared backbone and sequential classification heads, the model captures confidence patterns over iterations in a tree-like structure. This design draws from ideas in multi-task learning [3] and iterative refinement [17], but uniquely integrates prediction confidence into the classification hierarchy itself.

3. Methodology

NCT (NCT) incorporate confidence patterns into classification through a shared neural backbone, multiple category-specific classification heads, and an iterative training process. The approach progressively refines the label space to separate samples based on their classification outcomes across iterations.

3.1. Shared Backbone

The backbone network consists of three fully connected layers (128, 64, and 32 units) with ReLU activations and a dropout layer after the final hidden layer. This backbone produces a representation $h(x) \in \mathbb{R}^d$ for input x , which is shared across all classification tasks. Across iterations, we reuse and fine-tune this backbone with a reduced learning rate while only replacing the classification heads. This approach preserves learned representations while allowing adaptation to the expanded label space.

3.2. Category-Specific Classification Heads

For multi-category classification with m categories, the model employs m classification heads:

$$p^{(j)}(x) = \text{softmax}(W^{(j)} h(x) + b^{(j)}), \quad j = 1, \dots, m,$$

where $W^{(j)}$ and $b^{(j)}$ are the weights and biases for the head corresponding to category j . Each head specializes in its assigned category, enabling independent evaluation of category-specific performance through accuracy metrics.

3.3. Iterative Refinement

NCT refines classifications over T sequential iterations. The output dimension of each classification head expands exponentially:

$$d^{(1)} = 2, \quad d^{(t)} = 2 d^{(t-1)}, \quad t = 2, \dots, T,$$

yielding $d^{(2)} = 4$, $d^{(3)} = 8$, and generally $d^{(t)} = 2^t$. The first iteration involves binary classification for each category. Between iterations, the classification heads are reinitialized with expanded dimensions to accommodate the larger label space created through the re-labeling process.

3.4. Tree-Structured Re-labeling

After each iteration, new labels are generated based on both true labels and model predictions:

$$\text{new_label} = \begin{cases} \text{pred} \times 2 & \text{if true_label} = 0 \\ \text{pred} \times 2 + 1 & \text{if true_label} = 1 \end{cases}$$

This encoding creates four distinct paths after the first iteration (0,1,2,3), reflecting both ground truth and prediction correctness. For example, index 0 represents samples with true label 0 correctly predicted as 0, while index 2 represents samples with true label 0 incorrectly predicted as 1. When applied recursively across iterations, this encoding creates a hierarchical tree structure where even indices generally correspond to paths with previous misclassifications, and odd indices correspond to paths with correct predictions.

In practice, when we switch from the old 2^{t-1} -way labels to the new 2^t -way pseudo-labels, we still supervise the backbone with the original binary ground truth. Specifically, we compute two loss terms at each training step: $\mathcal{L}_{\text{pseudo}}$, a cross-entropy loss using the current iteration's expanded labels; and $\mathcal{L}_{\text{true}}$, a secondary cross-entropy computed by collapsing the logits of the 2^t -way head into two bins (even vs. odd indices) and comparing against the binary label. We combine the two as $\mathcal{L} = \mathcal{L}_{\text{pseudo}} + \alpha \mathcal{L}_{\text{true}}$, with a small α (e.g., 0.1), ensuring the model retains sensitivity to the original label and can correct earlier misclassifications.

Table 1: Performance comparison across datasets

Dataset	Our Model	DNDT	DT	NN
Iris	90.0	100.0	100.0	100.0
Haberman’s Survival	75.8	70.9	66.1	70.9
Car Evaluation	98.7	95.1	96.5	91.6
Titanic	83.8	83.8	79.0	76.9
Pima Indian Diabetes	77.27	66.9	74.7	64.9
HR Evaluation	91.72	92.1	97.9	76.1
Breast Cancer Wisconsin	97.5	94.9	93.0	77.5

<i>Results on CelebA dataset</i>	
Model	Accuracy (%)
NCT	83.73
CNN	84.14

3.5. Inference

During inference, input samples are processed through the backbone and all classification heads. For each category’s classification head, a confidence score is computed by summing the probabilities at the “positive” indices:

$$\text{score}_j = \sum_{i \in \mathcal{I}_{\text{pos}}} p_i^{(j)}(x)$$

where $\mathcal{I}_{\text{pos}}$ represents the set of indices corresponding to paths with correct predictions in the final iteration (typically the odd indices, e.g., $\{1, 3, 5, 7\}$ for $T = 3$). The final prediction is the category corresponding to the classification head with the highest score.

The algorithm of the model is provided in Appendix A.1 for reference.

4. Experiments

We evaluate our NCT against baseline methods on a suite of benchmark datasets. Table 1 summarizes the test accuracy (%) of NCT compared to Deep Neural Decision Trees (DNDT) [18], standard decision trees (DT) [2], and a feed-forward neural network (NN) [14]. Table 5 lists all the hyperparameters used for our NCT. Hyperparameter analysis is provided in Appendix A.3.

4.1. Classification Performance Analysis

Our NCT framework demonstrates competitive performance across datasets, notably outperforming the standard neural network baseline in all cases. Particularly significant improvements are seen on the Pima Indian Diabetes dataset (+10.37% over DNDT) and Breast Cancer Wisconsin (+2.6% over DNDT, +4.5% over DT). These results suggest that incorporating confidence patterns into the classification structure effectively captures informative features that conventional approaches might miss.

We additionally evaluate our model on the CelebA dataset using a lightweight CNN backbone (see Appendix A.4 for details). As shown in Table 1, our model achieves 83.73% accuracy—closely

matching the baseline CNN performance of 84.14%. This confirms the model’s competitiveness on high-dimensional image data while retaining interpretability

5. Interpretability Framework

NCT offer inherent interpretability by encoding confidence patterns into the classification structure. The hierarchical label space reveals how samples distribute across confidence levels.

For each test input x , we compute a difficulty score $\Delta(x) \in [0, 1]$ that reflects both prediction confidence and correctness. Let $\mathbf{p}^{(j)}(x)$ be the softmax output from classification head j , and define $s_j(x) = \sum_{i \in \mathcal{I}} p_i^{(j)}(x)$, where $\mathcal{I}$ contains indices for “positive” classes (e.g., $\{1, 3, 5, 7\}$). The dominant head is $j^*(x) = \arg \max_j s_j(x)$.

From this head, the most confident positive index is $i^*(x) = \arg \max_{i \in \mathcal{I}} p_i^{(j^*(x))}(x)$, with confidence $\rho(x) = p_{i^*(x)}^{(j^*(x))}(x)$. The model’s prediction is $\hat{y}(x) = \arg \max_k p_k^{(j^*(x))}(x)$, and correctness is $\delta(x) = \mathbf{1}\{\hat{y}(x) = y(x)\}$. The difficulty score is defined as $\Delta(x) = 1 - \rho(x)$ if correct, and $\Delta(x) = \rho(x)$ if incorrect.

Thus, $\Delta(x) = 0$ for confident correct predictions, $\Delta(x) = 1$ for confident errors, and intermediate values reflect uncertainty. Appendix A.2 provides further intuition and extraction details.

6. Conclusion

NCT establish a unique approach to classification by structurally encoding prediction confidence through progressive label expansion. By leveraging a shared backbone and category-specific heads that distribute samples across a hierarchical label space, the framework naturally separates consistently high-confidence predictions from challenging cases. Experimental results on benchmark datasets demonstrate competitive performance across all classification tasks, notably achieving 77.27% accuracy on Pima Indian Diabetes and 97.5% on Breast Cancer Wisconsin. With only a 7% parameter increase over standard networks, NCT offers an efficient framework for integrating confidence awareness into neural classifiers. This approach suggests that incorporating prediction uncertainty directly into model architecture may complement traditional uncertainty estimation techniques while maintaining competitive performance.

References

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- [2] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth International Group, 1984.
- [3] Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.
- [4] Andrey Figurnov, Michael B. Collins, Yukun Zhu, William Fedus, C. Lawrence Zitnick, Trevor Darrell, and Alexander Kolesnikov. Spatially adaptive computation time for residual networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1039–1048, 2017.

- [5] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.
- [6] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1050–1059, 2016.
- [7] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70, pages 1321–1330. PMLR, 2017.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [9] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017. URL <https://arxiv.org/abs/1610.02136>.
- [10] Gao Huang, Danlu Chen, Tianhong Li, Felix Wu, Laurens van der Maaten, and Kilian Q. Weinberger. Multi-scale dense networks for resource efficient image classification. In *International Conference on Learning Representations (ICLR)*, 2018.
- [11] Yani Ioannou, Duncan Robertson, Darko Zikic, Peter Kotschieder, Jamie Shotton, Matthew Brown, and Antonio Criminisi. Decision forests, convolutional networks and the models in-between. In *arXiv preprint arXiv:1603.01250*, 2016.
- [12] Peter Kotschieder, Madalina Fiterau, Antonio Criminisi, and Samuel Rota Buló. Deep neural decision forests. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1467–1475, 2015.
- [13] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pages 1097–1105, 2012.
- [14] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [15] J. Ross Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.
- [16] Swabha Swayamdipta, Roy Schwartz, and Noah A. Smith. Dataset cartography: Mapping and diagnosing datasets with training dynamics. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9275–9293, 2020.
- [17] Ryota Tanno, Nguyen Khac Bui, Daniel E Worrall, Enrico Kaden, Benjamin Schott, Jianfeng Zhang, Daniel C Alexander, and Yee Whye Teh. Dynamic tree generation for interpretable neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 1588–1596, 2019.
- [18] Yongxin Yang, Irene Garcia Morillo, and Timothy M. Hospedales. Deep neural decision trees. *arXiv preprint arXiv:1806.06988*, 2018.

Appendix A. Technical Details and Supplementary Results

A.1. NCT Algorithm

Algorithm 1 presents the complete training and inference procedure for NCT. The algorithm implements the iterative refinement process described in Section 3.3, including the tree-structured re-labeling mechanism that creates expanded label spaces across iterations.

Algorithm 1: NCT: Iterative Training and Inference

Data: Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, number of categories m , total iterations T , epoch schedule $\{E_t\}_{t=1}^T$, initial output dimension $d_0 = 2$

Result: Trained NCT model

Set random seed for reproducibility;

Preprocess $\mathcal{D}$ (handle missing values, normalization, encoding);

Split $\mathcal{D}$ into training set $\mathcal{D}_{\text{train}}$ and test set $\mathcal{D}_{\text{test}}$;

Prepare initial labels for each category-specific head;

Build an initial model with a shared backbone and m classification heads, each with output dimension 2;

for $t = 1$ **to** T **do**

 Train the current model on $\mathcal{D}_{\text{train}}$ for E_t epochs;

 Obtain predictions $\hat{y}^{(t)}$ for each head on $\mathcal{D}_{\text{train}}$;

for each category $j = 1$ **to** m **do**

for each training sample i **do**

if $y_{j,i} = 0$ **then**

 Set new target label for head j : new label $= \hat{y}_{j,i}^{(t)} \times 2$;

else

 Set new target label for head j : new label $= \hat{y}_{j,i}^{(t)} \times 2 + 1$;

end

end

end

 Update the model by replacing each classification head with a new head having increased output dimension;

 Fine-tune the shared backbone with a reduced learning rate;

end

Inference:

for each test sample x **do**

for each category $j = 1$ **to** m **do**

 Compute head outputs: $p^{(j)}(x) = \text{softmax}(W^{(j)}h(x) + b^{(j)})$;

 Compute score for category j : $s_j(x) = \sum_{i \in \mathcal{I}_{\text{pos}}} p_i^{(j)}(x)$;

 where $\mathcal{I}_{\text{pos}}$ contains indices for correct predictions (typically odd indices);

end

 Final prediction: $\hat{y}(x) = \arg \max_j s_j(x)$;

end

return Test accuracy and difficulty scores $\Delta(x)$

A.2. Interpretability Framework

The difficulty score $\Delta(x)$ provides a unified metric for assessing prediction confidence and correctness. This score captures both the magnitude of confidence and its alignment with ground truth in a single scalar value.

A.2.1. INTUITION AND FORMULATION

The score $\Delta(x)$ attenuates correct predictions by their confidence margin $(1 - \rho)$ and accentuates confident errors by retaining ρ . This creates a continuous measure that captures both confidence magnitude and its alignment with ground truth, providing an instance-level difficulty metric.

The score smoothly interpolates between values based on model confidence and correctness:

- $\Delta(x) \approx 0$: Highly confident correct prediction
- $\Delta(x) \approx 0.5$: Uncertain or borderline case
- $\Delta(x) \approx 1$: Highly confident incorrect prediction

A.2.2. EXTRACTING HIGH AND LOW CONFIDENCE EXAMPLES

Algorithm 2 demonstrates how the difficulty score can be used to systematically separate samples into high and low confidence sets based on a threshold parameter.

Algorithm 2: Extract High- and Low-Confidence Examples

Input: Test inputs $\{x_k\}$, difficulty scores $\Delta(x_k)$, threshold $\tau \in [0, 1]$

Output: High-confidence set $\mathcal{C}$, Low-confidence set $\mathcal{L}$

Initialize $\mathcal{C} \leftarrow \emptyset$, $\mathcal{L} \leftarrow \emptyset$;

for each sample x_k **do**

if $\Delta(x_k) \leq \tau$ **then**

 Add x_k to $\mathcal{C}$

 Add x_k to $\mathcal{L}$

end

return $\mathcal{C}$, $\mathcal{L}$;

A.3. Hyperparameter Analysis

We conducted extensive hyperparameter tuning to optimize the performance of NCT across multiple datasets.

A.3.1. FINE-TUNING EPOCHS ANALYSIS

Table 2: Test accuracies (%) of NCT with different fine-tuning epochs

Dataset	Epochs = 10	Epochs = 20	Epochs = 30
Iris	90.0	90.0	90.0
Haberman’s Survival	74.2	75.8	76.2
Car Evaluation	96.7	98.7	98.7
Titanic	82.12	83.8	85.47
Pima Indian Diabetes	76.5	77.27	74.68
HR Evaluation	91.59	91.72	91.57
Breast Cancer Wisconsin	96.5	97.5	98.1

Table 2 presents the effect of varying final-layer fine-tuning epochs on model performance. Most datasets show improvement when increasing from 10 to 20 epochs, with Car Evaluation and Breast Cancer Wisconsin showing the most substantial gains. However, extending to 30 epochs yields mixed results, with Pima Indian Diabetes actually showing performance degradation, suggesting potential overfitting. Based on these findings, we selected 20 epochs as the standard configuration for our experiments.

Table 3: Accuracy (%) for varying epoch counts in Iteration 1 and Iteration 2

Iter 1 Epochs	Iter 2 Epochs	Car Eval	BCW	Haberman	Iris	Pima Diabetes	Titanic	HR Eval
1	4	98.0	96.5	59.2	83.3	74.58	84.36	91.42
1	5	98.0	97.5	72.7	86.7	74.77	83.24	91.48
1	6	97.1	97.5	74.0	83.3	75.00	86.03	91.36
2	4	98.5	97.5	75.2	91.2	77.27	84.36	91.58
2	5	98.5	97.5	74.0	86.7	76.67	86.03	91.57
2	6	98.5	95.6	63.6	83.3	74.58	84.92	91.68
3	4	98.8	97.5	74.0	83.3	74.68	85.47	91.71
3	5	98.5	97.5	74.0	83.3	75.58	84.92	91.72
3	6	98.8	96.5	74.0	86.7	75.68	84.92	91.68

Table 3 provides a more detailed analysis of epoch sensitivity across the earlier iterations. We observed that the combination of 2 epochs for Iteration 1 and 4 epochs for Iteration 2 yielded the best overall performance across datasets, with optimal results for Breast Cancer Wisconsin, Haberman’s Survival, Iris, and Pima Indian Diabetes. While Car Evaluation achieved slightly higher accuracy with 3 epochs in Iteration 1, and Titanic and HR Evaluation showed marginal improvements with other configurations, the (2,4) combination provided the best balance of performance and computational efficiency.

A.3.2. BATCH SIZE ANALYSIS

Table 4: Test accuracies (%) of NCT with different batch sizes

Dataset	Batch Size 16	Batch Size 32
Iris	90.0	90.0
Haberman’s Survival	75.8	74.2
Car Evaluation	96.7	96.5
Titanic	83.8	80.45
Pima Indian Diabetes	77.27	72.08
HR Evaluation	91.72	91.68
Breast Cancer Wisconsin	97.5	97.5

Table 4 compares model performance with two different batch sizes. The smaller batch size (16) consistently outperforms or matches the larger batch size (32) across all datasets.

A.3.3. TREE DEPTH ANALYSIS

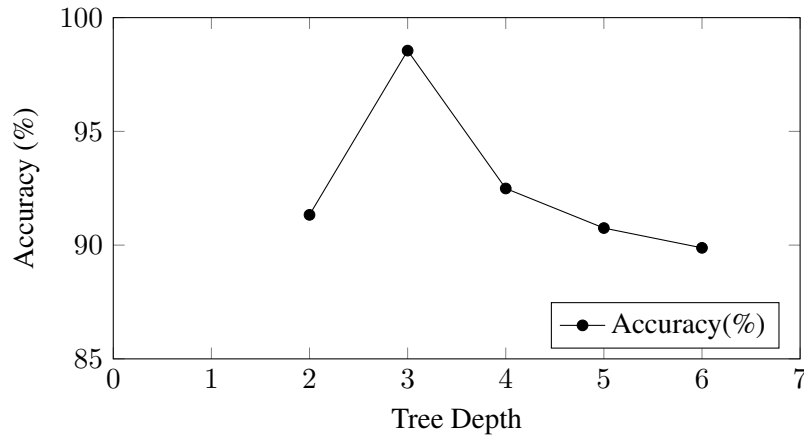


Figure 2: Test accuracy as a function of tree depth (number of iterations)

Figure 2 illustrates how model performance varies with tree depth. The accuracy peaks sharply at depth 3 (98.55%) and consistently declines with deeper trees.

A.3.4. FINAL HYPERPARAMETER CONFIGURATION

Table 5: Hyperparameters used for NCT experiments

Hyperparameter	Value
Input features (d)	dataset-dependent
Shared dense layer sizes	128, 64, 32
Number of heads (m)	dataset-dependent
Number of iterations (T)	3
Output dimensions per iteration	$\{2, 4, 8\}$
Batch size	16
Initial learning rate (Adam)	1×10^{-3}
Fine-tuning learning rate	1×10^{-4}
Epochs (iteration 1)	2
Epochs (iteration 2)	4
Epochs (iteration 3)	20
Train/test split	80% / 20%

Table 5 summarizes the full set of hyperparameters used in our experiments. Note that the epoch schedule increases with each iteration, reflecting the increasing complexity of the classification task as the label space expands. The learning rate for fine-tuning is reduced by an order of magnitude compared to the initial training phase to prevent catastrophic forgetting of learned representations while allowing adaptation to the expanded label space.

A.4. Backbone CNN Architecture

The shared backbone is a lightweight convolutional neural network composed of three convolutional blocks. Each block consists of a convolutional layer (kernel size 3, padding 1), followed by a ReLU activation and a downsampling step using either max pooling or stride. The channel dimensions increase across layers: 32, 64, and 128. The final feature map is globally averaged and passed through a fully connected layer to produce a compact latent representation of dimension 128. This feature vector $h(x)$ is shared across all classification heads and forms the basis for prediction and relabeling across iterations.

A.5. Parameter Efficiency Analysis

Table 6: Parameter counts: FNN vs. NCT

Model	Total Parameters	Example ($D = 100, m = 8$)
FNN (single head)	$\underbrace{P_{\text{base}}}_{=128D+10,464} + 33m$	23 200
NCT (m heads of size d)	$P_{\text{base}} + 33md$	$d = 4 : 23\,928$ $d = 8 : 24\,856$

Table 6 compares parameter counts between a standard feed-forward network (FNN) and our NCT, both using identical $\{128, 64, 32\}$ backbones. With 8 category-specific heads of dimension $d = 8$, NCT requires only 7% more parameters than FNN (24,856 vs. 23,200), demonstrating that our tree-structured approach delivers confidence-driven decision capabilities with minimal computational overhead.